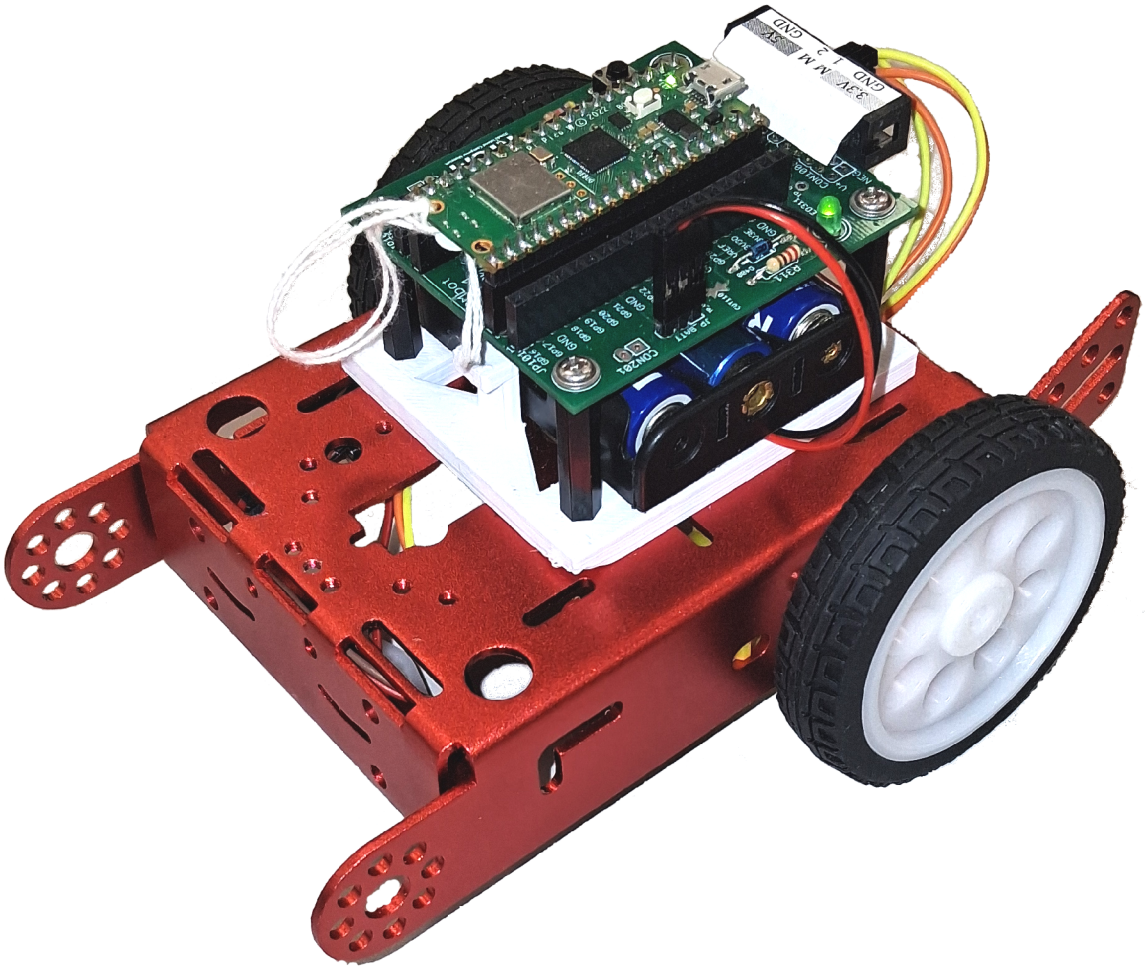
RICM CarBot

(ricomputermuseum.org)



Goal:

Create a modifiable 2 wheeled robot capable of currently 4 functions:

- Ability to be remotely controlled using any web browser
- Autonomous operation by collision avoidance
- Ability to follow a thick black line
- Ability to travel in a programmed pattern.

The robot is to be built by students from age 8 on up using standard mechanical components, screws, motors, plastic components. Students will also learn how to solder a custom-built printed circuit which will be the basis of expandability and provide support circuitry to control motors and other sensors. You can switch modes with a small push button located on the custom printed circuit board. An onboard green LED flashes to indicate the selected modes. You can change mode anytime.

Hardware:

The “brains” of the robot car is a pre-programmed Raspberry Pi Pico W. The Pico was selected due to its ability to avoid program corruption when power is removed, and its size. RICM needed to have a board that the Pico could plug into that also had an onboard motor controller capable of driving wheel motors from 2.5 – 12 volts at 800ma of current per motor. No readily available boards were located that fit our needs, so RICM developed their own PCB. This custom PCB has mounting place in the middle for the Pico, (2) L9110 motor drivers, a power LED, a pushbutton. It also includes power pins for 3.3 V and 5 V as well as associated grounds to provide power to other devices such as sensors, lights etc.

The board uses GPIO pins for hardware interfacing, GPIO-14 for the pushbutton, GPIO-18-19 for the right motor and GPIO-20-21 for the left motor, for a total of 5 GPIO pins, leaving many more GPIO pins for expansion & experimentation. No external wires are needed to connect the motor drivers to the Pico as they are permanently assigned on the PCB.

On the Pico, there are also (3) analog to digital converters (ADC). The Pico uses these to convert analog signals or varying voltage, represented as numbers (0-4096) to be used for working with sensors that operate with analog voltages. These ADCs need a *reference voltage* to set the analog voltage against. Therefore, the custom PCB connects

the regulated 3.3 V of the Pico to the reference voltage of the ADCs. so the ADCs can convert analog voltages from 0 to 3.3 V to a number between 0-4096! The reference voltage *can* be changed by cutting a bowtie looking PCB trace and supplying the ADCs with another voltage reference. The new voltage reference *must* be *very* stable, so it is not recommended this should be done, unless you know what you are doing.

The motors will operate from 1.5 to 6 V, thus making for easy & simple battery operation. The car uses (3) AA batteries producing 4.5 V to the board and motors. The Pico needs 1.8 to 5.5 V to operate, so this battery pack is ideal for both the Pico and the motors.

The custom PCB mounts to a 3D-printed adapter plate, that is raised up so battery pack can slide under it. Software is all open source and can be modified using standard Python language at will by the student, or used as-is. All hardware, except the custom motor driver board, is readily available, as off-the-shelf parts.

Software:

With the small exception of HTML & JavaScript used for the *remote control* section, and *pattern upload* section, the software uses no special libraries and uses standard Python programming language. We chose Python because it is a good beginners programming language. All software is included with the robot and can be easily modified using the Thonny development environment. Thonny is a free software environment that is easily loaded onto any PC (Mac, Linux, Windows). Users should be familiar with this application and Python.

The robot has 4 modes it can operate in:

1. Remote control (default)
2. *Autonomous collision
3. Programmed patterns
4. *Line following

* - Extra hardware required

You can select the mode with the pushbutton located on the RICM motor control board. Press it once and wait for blinking light to turn solid, then go off. Press the button *X* times, were *X* is the mode you want the robot to be in. Once selected, the LED on the Pico will flash the mode it is in. If you select a mode that the hardware does not support, or external hardware is missing, the LED will flash rapidly. You can change modes anytime.

OPERATION

When power is applied, the green LED on the RICM Driver board will light up indicating that power is applied. The green LED on the Pico will blink slowly indicating the default mode it is in.

To change the mode, press the push button on the RICM Driver board once. The Pico will respond with a solid light, then it will switch off. At this point, press the button to set the mode: press it 3 times for mode 3, once for mode 1 etc. The Pico will now flash the LED to indicate the current mode.

> **NOTE:** the mode will reset to the default mode each time you apply power. This default mode can be set in the *globalv.py* module in the *lib/* directory using Thonny.

Testing motor connections

The motor connections are the four center pins in the plastic connector at the rear of the car. Make *sure* the wires are connected on those four center pins or the motors will not work properly. (see Figure-1) M1 is for motor 1 and M2 is for motor 2.

Since these motors have no polarity, it's a simple task to ensure motors are plugged in correctly. Use the *remote* mode and move the joystick fully forward. Both wheels should spin in a forward direction. If not, simply reverse the wires going to the motor that is spinning in the wrong direction. Once you get the forward and backward motion correct. Then test left and right wheels for proper direction of the robot. If direction is wrong, simply swap both motor wires while maintaining their orientation. Once this configuration is set, it never has to be changed. If you unplug the motors and forget where they went, you will have to re-do this step.

3v3	3v3	3v3	3v3	M1a	M2a	5v	5v	5v	5v
Gnd	Gnd	Gnd	Gnd	M1b	M2b	Gnd	Gnd	Gnd	Gnd

Figure-1. Plastic connector pin assignments

ASSEMBLY

( ) 1. Make sure you have all the materials listed in the Bill of Materials (BOM) first

Tools Needed:

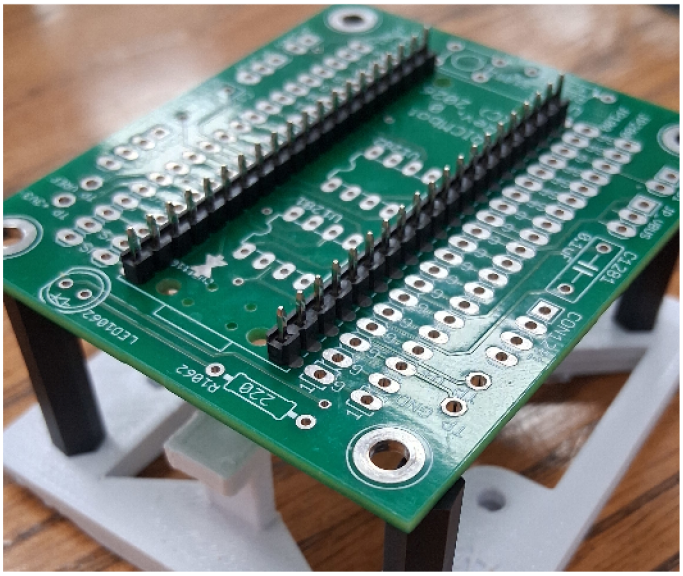
- small Phillips screwdriver
- small wire cutter
- small soldering iron & solder

Soldering

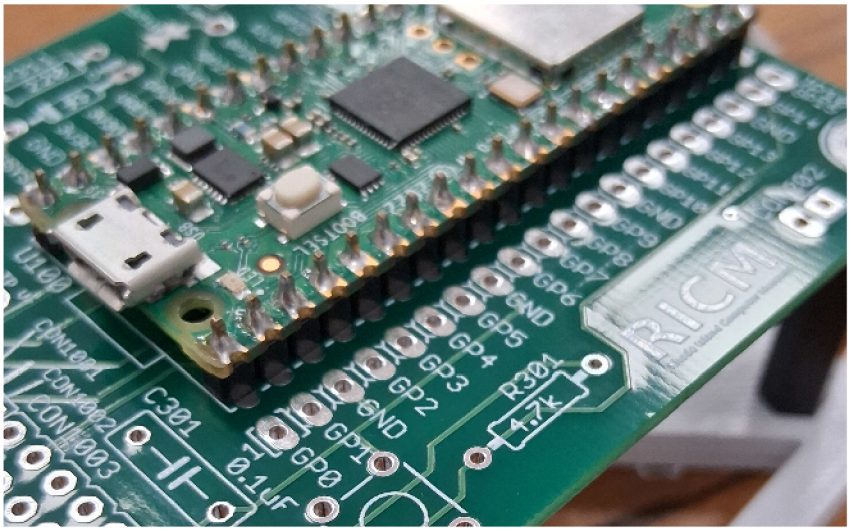
NOTE: Check off each step as you proceed

Skip to step 4 If you already have a Pico-W with male headers attached.

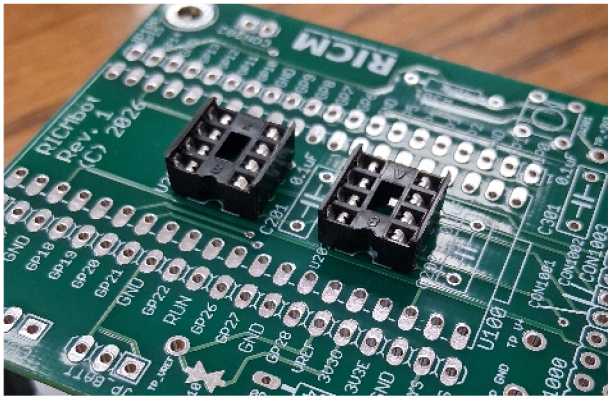
( ) 2. Place the long pins of the male headers into the Printed Circuit Board (PCB) where the Raspberry Pi Pico would go. **DO NOT SOLDER THE PINS TO THE BOARD!!**

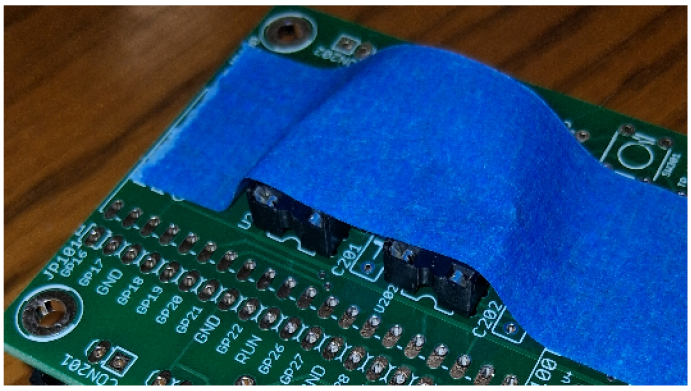


( ) 3. Place the Pico onto the short pins of the header. Solder the first and last pin on each header. Make sure the headers are perfectly aligned and perpendicular with the Pico. At this point you can remove the Pico with the male headers and finish soldering the headers to the Pico. Inspect all the solder joints before continuing.

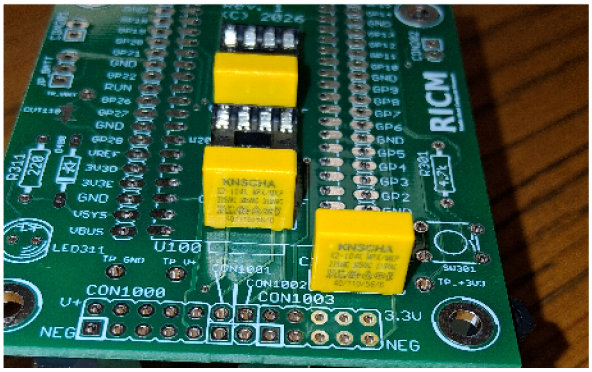


( ) 4. Solder the 8pin DIP sockets to the PCB, pay attention to the orientation of the sockets, such as the notch of the socket matches the picture on the PCB. You can use masking tape to hold the sockets in position if you need to.

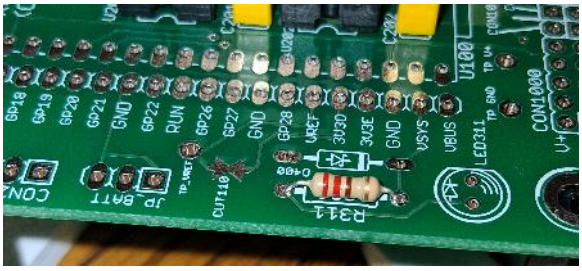




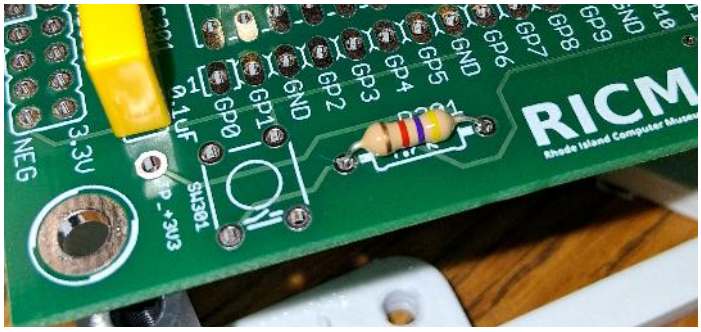
( ) 5. Solder the yellow capacitors into C201, C202, and C301. Make sure they are flush with the board. Clip off the excess leads from the bottom of the board.



( ) 6. Solder the 220 ohm (Red-Red-Brown) resistor into R311. Make sure it is flush with the board. Clip off the excess leads from the bottom of the board.

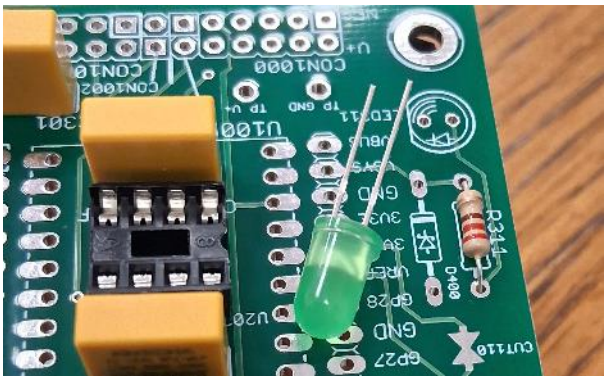


( ) 7. Solder the 4.7k ohm resistor (Yellow-Purple-Red) resistor into R301. Make sure it is flush with the board. Clip off the excess leads from the bottom of the board.



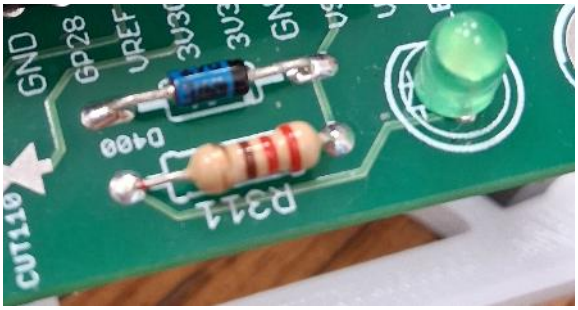
( ) 8. Solder the LED into LED311. All Diodes are polarity sensitive, so it's important that they are installed correctly. Failure to install correctly will result in the LED not working. Long leg is USUALLY the Anode or "+" side of LED. "+" side of LED is denoted on the left side of this symbol ▶|Clip off the excess leads from the bottom of the board.

Hint: Before soldering, put LED in place and plug in battery, if it lights, it has the proper orientation, if not, reverse the direction of the LED. REMOVE the battery before soldering





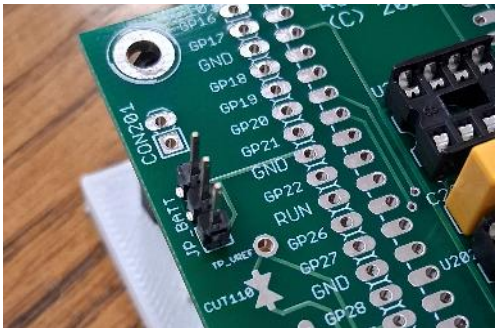
( ) 9. Solder the blue diode into D400. The black line on the diode needs to match the line on the PCB. If this is installed wrong, the board will not get power. Make sure it is flush with the board. Clip off the excess leads from the bottom of the board.



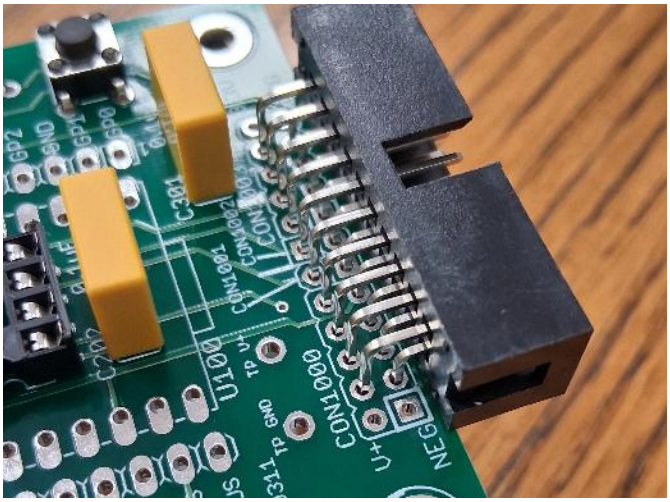
( ) 10. Solder the pushbutton into SW301. NOTE: it only goes in one way.



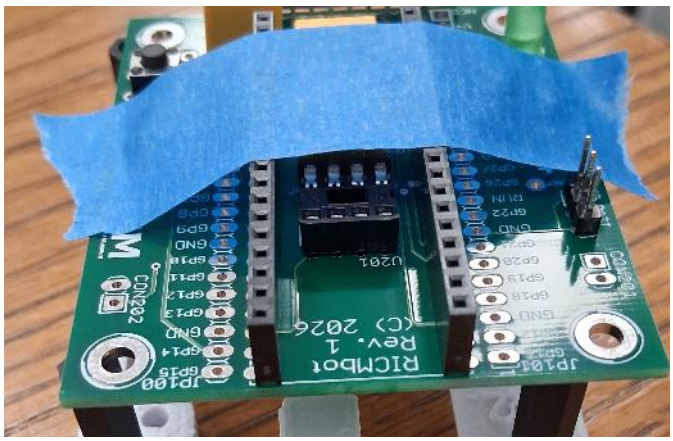
( ) 11. Solder the 3-pin male header into JP_BATT. The short end of the pins goes into the PCB.



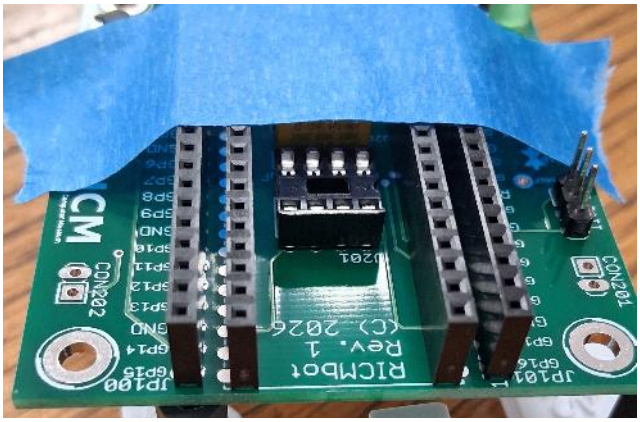
( ) 12. Solder the right-angle IDC connector. Leave the 1st and last position empty.



( ) 13. Solder the Pico female headers. Insert the headers onto the PCB, and hold them down with tape. Only solder the first and last pin at first to make sure they are aligned properly. After the two pins on each side are soldered in place and they are upright, finish soldering the headers in place.

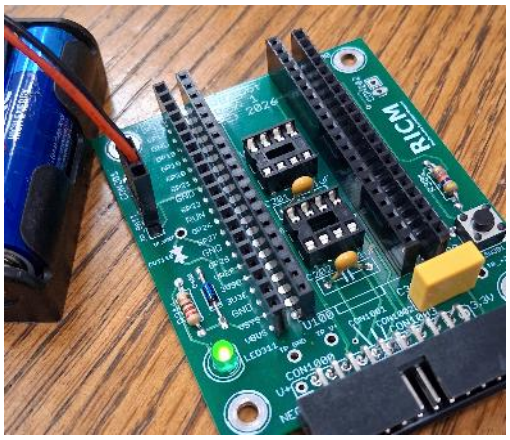


( ) 14. Solder the last two female headers next to the Pico headers. Use masking tape to hold them in place when you solder. Again, just solder the first and last pin and check for alignment *before* soldering the rest of the pins.



DO NOT INSTALL THE PICO OR MOTOR DRIVER CHIPS INTO THE SOCKETS

At this point you may connect that battery pack to JP_BATT. The LED should light. If the LED lights, you may continue. If not, remove the battery and check all soldering joints.



( ) 15. Strip off and tin about 1/8” of the insulation off each end of the female Dupont wires.

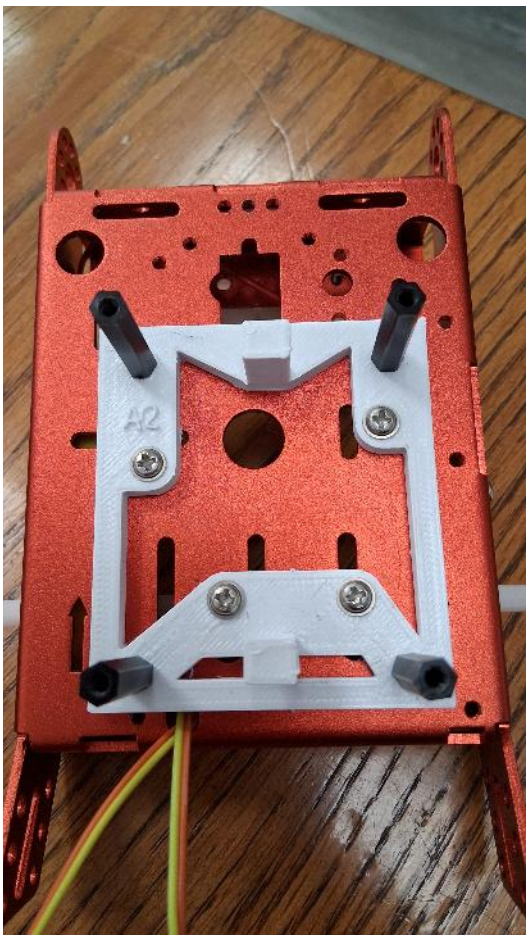
( ) 16. Remove the clear rubber hold down at the end of the motors and solder a wire to each of the motor terminals. Replace rubber hold down after soldering. Be careful not to overheat the motor connections.

Mechanical Assembly

( ) 17. Attach the 4 plastic posts to the top of the A2 plate using (4) 8mm screws. Be careful **not** to over tighten the screws, otherwise you will strip the threads in the posts.



( ) 18. Attach the plate to the rear of car chassis using (4) 8mm screws



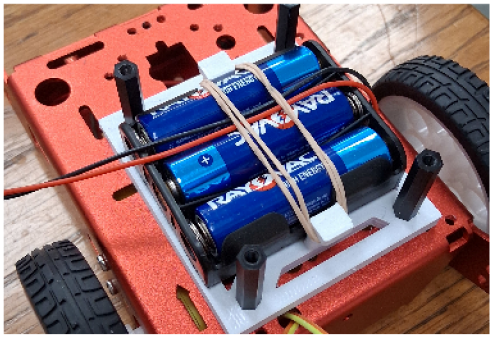
( ) 19. Attach each motor with long screws & nuts **(picture)**

( ) 20. Press the Wheels onto motor shafts

( ) 21. Attach the metal guide ball to the front of the chassis using two screws

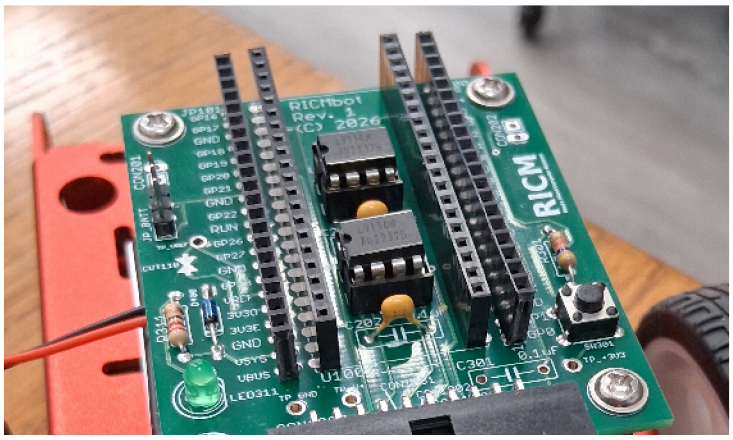
( ) 22. Guide the motor wires up through the chassis and through the small round hole at the back of the chassis.

( ) 23. Place the battery pack between the two guideposts and place a rubber band between the two posts to keep the battery in place.

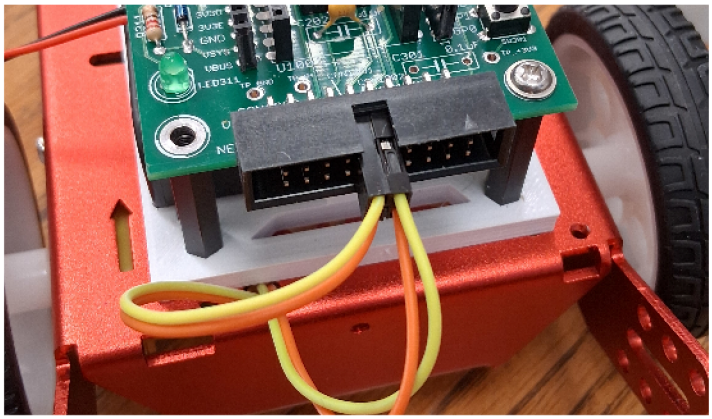


( ) 24. Attach the PCB to the four plastic posts with four 8mm screws.

( ) 25. Install the two motor driver ICs, pay *careful* attention to the notches in the ICs and sockets. Failure to install correctly will destroy the ICs. Make sure they are pressed all the way down. Inspect to determine whether any pins are not inserted fully.

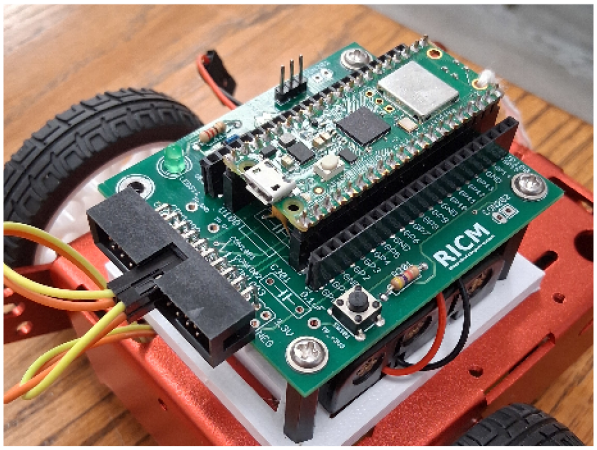


( ) 26. Plug the wires from the motors to the **center** of the IDC connector at rear of car (see Figure-1). At this point the polarity of the wires does not matter, as it can be corrected later.



( ) 27. Plug the Raspberry Pico into the female headers at the center of the PCB being careful to orient the Pico with the USB connection at the rear of the car chassis.

Make sure ALL pins on the Pico are seated in the female headers



DO NOT PLUG THE BATTERY IN YET!!!

Software install & configuration

Install MicroPython on the supplied Pico-W (do this only once)

Use a USB cable to connect the Pico-W to your computer

The Pico supplied is a Pico-W.

Go to Raspberry Pi website and follow instructions for installing MicroPython onto the Pico W:

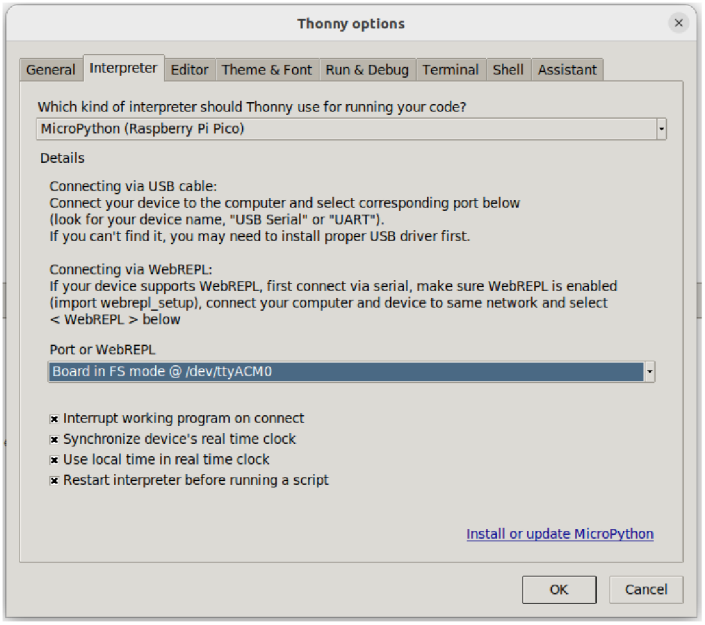
https://www.raspberrypi.com/documentation/microcontrollers/micropython.html#what-is-micropython

Install Thonny

goto Go to https://thonny.org/ to install Thonny for your computer (Windows or Linux)

Start Thonny

1. Select TOOLS – GENERAL (UI MODE). Select **regular mode**, exit and restart Thonny.
2. Click the **Tools** menu and select **Options**.
3. Under the **Interpreter** tab, ensure you make sure you have *Micropython Raspberry Pi Pico* selected, along with proper serial port.



4. Restart Thonny and you should see your Pico in the left column in the Files tab. If the Files tab is not visible, click the **View** menu and select **Files**. If not, check your setting again, make sure you are not using a USB charge only cable.

INSTALL RICM SOFTWARE

1. Use any Web Browser and type https://github.com/ricomputermuseum/CarBot
2. Download CODE (zip file) into your "Downloads" directory
3. Change to your "downloads" directory

WINDOWS:

Right click on the CarBot-main" folder (its a zip file)
Click on "Extract all" (accept default of CarBot-main folder)
This will create a CarBot-main folder in your Downloads folder
Open the "CarBot-main" folder (you have to do this twice)

Linux:

Navigate to you Downloads Directory
$unzip CarBot-main.zip -d ./ (this will unzip the files into a CarBot-main dir)
$cd CarBot-main

4. In the CarBot-main" folder there will be 1 file (README.md) and 2 folders (Documents & Software)
5. The documents folder has two imortant files: "RICM_CarBot_Manual.pdf" & "RICM CarBot BOM.pdf"
6. "RICM CarBot BOM.pdf" is a complete parts list (Bill of Materials)
7. "RICM_CarBot_Manual.pdf" is the build manual
8. Open Thonny (which you previously installed)
9. The left side of Thonny should have a files section. If not click on "View" at the top then select "Files"
10. Navigate to your "Downloads" Directory on your local computer, then to CarBot-main (Windows 2x) (Linux just once)
11. Navigate to the Software folder
12. You should see 3 files (main.py, Pattern.html, & remote.html) and 1 folder (libs)
13. Hightlight those 4 items (click on the "LIB" folder then mouse down to remote.html, hold the shift key and click the mouse once)
14. Right click and select "Upload to /" (this will copy all files to the Pico-W)
15. At this point all the software has been copied to the Pico and can now be configured

Configuration:

(using Thonny)

1. Click the Lib directory on the Pico
2. Double-click the *globalv.py* file, so you can edit the file
3. Edit the robot name. (Robot name MUST be unique if more than 1 robot in use)
4. Edit the default mode (currently – NOMODE)
   You should type in “REMOTE” as the default mode
5. Save the file
6. Exit Thonny

Next: Unplug the USB cable

1. Plug the battery onto the PCB
2. The LED on the Pico should blink the default mode its in, repeatedly, until changed via pushbutton.

MOTOR DIAGNOSTICS

See page-4 to properly test motor direction. Once this is done, it should not have to be done again

Have FUN with your new Car Robot!

THEORY OF OPERATION

REMOTE CONTROL (Mode-1)

In Mode-1, the default mode, the LED on the motor control board will flash once then a delay and repeat, indicating that it is in Mode-1. The name/password of the robot is stored in the *globalv.py* module. By default, the password is the same as the robot name, but can be changed in the *globalv* module.

In Mode-1, The robot will act as a WiFi access point, and the controlling device (i.e. laptop, iPad, phone, etc.) must be on this network. The SSID and password for the robot will be something like "Robotxxx" where xxx is a number like 001, 007 etc. This can be modified through software changes. The password for the access point is configured to be the same as the name you see when trying to connect. The XXX is written on the Pico, so it does not need to be remembered. Once connected, simply open a browser and type in "*robot.run*" in the URL line. Only one user at a time may connect with their browser to allow exclusive control of the robot. Once connected, the user will be presented with an onscreen Joystick, allowing you to move the robot using a simple finger movement. Once you take your finger off the screen, the joystick will go to a neutral position and stop the robot. This is a fun way to control the robot.

Detailed operation:

The *main.py* module calls the REMOTE_MODE function in the *remote.py* module. This function should return a code of either a 0 (normal) or a negative number, indicating an error.

Once the REMOTE_MODE function is called, it will set up an access point, DNS server, and web server in the *ws_stuff.py* module. This module is responsible for all WiFi activity. When the services are activated, the function waits for a connection from a browser. Typically when a browser starts, it looks for a captive portal page. This has to be ignored via the DNS server to allow the user to type in a URL. The URL name that is used is stored in the *globalv.py* module. When the URL is typed in, the browser sends this to our DNS server, which resolves the name to the IP address of the Pico, where it connects to the HTTP server. The server will send the root document to the browser. This document is stored on the Pico root directory as *remote.html*. This web page must be limited to 6k in size. The web page also has some JavaScript (JS) associated with it. It is the job of the JS to handle communications from the joystick control to the *remote.py* module.

Since web traffic is VERY "chatty", normal web conversation won't be fast enough to handle the required speed. Therefore, the movement of the mouse was scaled to allow

for +10 and -10 steps, both forward, backward, side to side. When the data is sent to the Pico it is scaled back up to +100 to -100 in all four directions, because the motor driver expects movement as a percentage. This greatly reduced the web traffic that would have occurred. Web traffic uses TCP, which requires each packet of data received to be replied with an “ACK” before the next packet could be sent. Luckily the Pico TCP socket scheme handles this for us.

The “joystick” format is more intuitive to the user, so that’s the main reason why it was chosen. The user can close the browser anytime and the software will close the connection and wait again for another connection and start all over again. If the user presses the onboard push button on the PCB, the robot will shut down all services and allow the user to change the mode. This is a very simple mechanism as well.

The *remote.py* module requires the *ws_stuff.py*, *globalv.py*, and *return_codes.py* modules, in addition to the standard MicroPython *network*, *socket*, and *gc* libraries.

See the function listings for *remote.py* to see what each function does.

Pre-Programmed Patterns (Mode-2)

Mode-2, the LED on the Pico will flash TWICE times then a delay and repeat while in this mode, indicating it is in Mode-2.

This is another web-based module. Therefore upon selection of this mode, the *main* function will call *get_pattern* in the *pattern.py* module. This function will set up an Access Point (AP), DNS server (DS), and a webserver (WS). These functions are in *ws_stuff.py*. This module is also used with the *remote* module, so be careful if you modify it. Once the servers are active, the function waits for a web connection. Just like the remote mode, you first need to connect to the robot AP. Use the SSID name and password that is set in *globalv.py*. Without modification, the SSID and password are the same, typically used is RobotXXX, where XXX is a number like '007'. Your device may complain that there is no internet connection, which is fine for our purpose. Once your device has been connected to the AP, you need to open a browser to upload a pattern file to the robot.

When a browser is opened, it will attempt to search for a "captive portal", which our DNS will ignore. You type into the URL line on the browser, our URL which is "*robot.run*". This name is specified in the *globalv.py* module. If you modify this, do NOT use .com, .org .net etc. as your browser will want to connect via HTTPS instead of HTTP. HTTPS is way too complicated for the Pico to handle at this time. Upon successful connection, the browser will ask you for a file to upload using a standard file locating mechanism. Upon file selection, the browser will send the file a line at a time, checking each line for proper commands and format that is in the *cmd_table.py* module. If anything is incorrect with a command or format, the import will fail with an error that needs to be corrected. If the line is correct, it is encoded in an internal pattern string.

Once the file has been successfully imported, the robot will follow this pattern endlessly, until a different mode is selected or power is removed. Once the pushbutton is pressed, the pattern data on the Pico will be cleared and memory reclaimed. The pattern module is very modular, allowing for new commands to be added, modified.

The user can write their own pattern for robot to follow.

Patterns can be created in a separate text file and name it whatever you like. ie. *pattern.txt*. The file is a plain text file with 1 command per line. Comments may be added to file of after a command. Comments begin with a "#" character, same as Python comments. It will ignore blank lines

Current pattern commands:

SXXX = Speed (0-100)

*Once Speed setting is made, it stays that way until changed. Speed 0 = stop

WXXX = Wait in Milliseconds

L = Left turn

R = Right turn

F = Forward

B = Backward

BBn = Begin a block. (n is 1-10)

EB = End a block definition

Exn = Execute a block (n is the block number)

Sample file for a square:

```
# ------------------- Define a block ----------------
BB6     # begin block 6
S100    # set speed to 100%
F       # move fwd
W250    # wait 1000ms  (1 second)
S010    # Set speed to 10%
R       # right turn
W25     # wait 25ms. This is how long the wheels stay in this turn
S0      # stop
EB      # end this block - 6
# ------------------ end this block --------------------
# define square block
EX6   # turn 1
EX6   # turn 2
EX6   # turn 3
EX6   # turn 4
w2000 # wait 2 seconds after 4 turns
#------------------------ Repeats pattern file endlessly
```

By modyfing the *cmd_table.py* it is possible to add more commands, such as turn on/off headlights, turn on/off turn/brake lights etc.

Autonomous collision avoidance (Mode-3)

Mode-3, the LED on the Pico will continuously flash THREE times, then a delay and repeat while in this mode, indicating it is in Mode-3.

The robot will move along the ground at full speed, on its own, sensing any obstruction via an ultrasonic sensor, then as it gets close to an obstruction, it will slow down, giving the opportunity for the obstruction to move away, if not, the robot will stop, back away, and turn away from the obstruction, then continue on its journey in a forward motion. This mode will continue forever until changed via push button.

The Collision Avoidance (CA) module requires extra hardware and mounting material, such as a 3.3 V ultrasonic sensor, mounting adapter and wires to plug into the expansion headers. If this sensor is not present, the onboard LED will flash rapidly. During operation in this mode, the sensor sends out ultrasonic (40 kHz) sounds and listens for reflected sounds. The time difference from when the sound was sent, to the time it is sent back, indicates how far the object is. The intensity or “loudness” of reflected sound, indicates how large an object is. This is typically called a “ping” just the same way submarines detect objects in the water.

Line Following (Mode-4)

Will be implimented in next release

Mode-4, the LED on the custom printed circuit board will flash *four* times then a delay and repeat while in this mode, indicating it is in Mode-4. In this mode, the robot will follow a black line drawn on a light background.

TECHNICAL DOCUMENTATION

GENERAL: As mentioned in the operation part of this manual, most of the software is written in standard MicroPython that runs in the Raspberry Pi Pico W. No special libraries or dependencies are used. The software is modular by design, lending itself to easy modification. A program called Thonny is used to program the Pico. Thonny is a free program that runs on Apple, Windows, or Linux computers. The Pico is attached via USB cable to the computer.

Any setup variables such as the robot name, default startup mode, etc. are in a file called *globalv.py* located in the /*lib* directory on the Pico. There is also a **DEBUG** variable which can be set to **True**, to aid in debugging the code, through various print statements to the console.

Using the custom RICM motor driver board, an import in *main.py* must import ricmboard, as Platform, the *ricmboard* will import *L9110* as the motor driver. The motor driver will actually control the motors.

Once a Platform and Motor have been established, the software will know how to control the robot. As per standard Pico operations, as soon as power is applied to the Pico, *main.py* will be executed.

Different modes of the robot are selected via the onboard pushbutton of the RICM board. Press it ONCE to enter the change mode. This mode will be indicated on the Pico as the LED stayig on for 2 seconds, followed by going off. You then press the button for the mode you want to be in. ie press 3 times for mode 3, etc. after you stop pressing the button, the Pico will enter that mode and constantly flash the mode number while it is in that mode. Any software that is running, needs to poll the “button_pressed” variable on the *globalv.py* module. If this variable is True, you must clean up your code, stop any motors, and just return to *main.py* with a return code of 0 (NORMAL). Any negative value will be deemed an error of some sort, where as any positive value would indicate some sort of status value.

Under Mode-1 (remote control) and Mode-2 (pre-programmed pattern control), WiFi and Web Servers are implemented. Using these two modes, some small JavaScript code is used to run on the users web browsers and is sent along with the web pages sent. In order to use these modes your device must be connected to the Access Point(AP). The name and password for the AP is specified in *globalv.py.* Once you have connected to the AP, simply open a browser and type in the URL name that is specified in *globalv.py*, the default is “robot.run”. Once you are connected, you will be presented with a joystick for remote control in the remote control mode, or a file upload form to upload a pattern to follow in the pattern follow mode.

The software lends itself to be very modular, so features can be added or removed without breaking everything else. You can run it on different motor chassis, using different motor controllers etc. that is the basic function of the car robot. For example if you want to use a different motor controller such as a stepper motor, you would need to remove the L9110 chips, wire up you motors to your controller and plug your wires into the Pico headers, write your own motor controller driver and import it into *ricmboard.py.* The controller only requires two functions, init() and drive(d,t). Also, you need to name your driver. D is the difference between left & right motor, and t is throttle.

MAIN.PY

This is a small module that runs on power up. It sets up the base environment, and passes control to the different modes, which is either the default mode, set in *globalv.py* or the mode that is set from *ricmmode.py*.

RICMMODE.PY

This module is responsible for blinking the LED as a regular timer interval regardless of what the application is doing. It blinks the Pico LED with the current mode. It also has the ability to make the onboard LED Flash rapidly in the event something goes wrong wit a mode setting. Ie setting the collision avoidance mode without the ultrasonic module being present, etc.

RICMBOARD.PY

This module is unique to the RICM motor driver board. It also loads the L*9110.py* module, as this is the actual code that talks to the motors. This module also sets up the pushbutton interrupt handler. This pushbutton is unique to the RICM Board. The Interrupt handler just sets the *button_pressed* variable located in *globalv.py*. This variable needs to be checked by any application and stop operation and return back to the *main.py* for proper mode handling. The actual mode switching is performed in the *ricmmode.py* module called from the *main.py* module.

L9110

This is the module that drives the motors. It has only two functions, init() initializes the driver if needed, and drive(d,t), actual driving of the motors. This module sets up Pulse Width Modulation (PWM), the GPIO Pins that are used, PWM Frequency, and Duty Cycle.

To make the motors move, the *drive* function is used. It takes two parameters, *d* and *t*. *d* is the differential, 0 to -100 for the left and 0 to 100 for the right. *t* is the throttle. 0 to 100 for a forward motion and 0 to -100 for reverse. For example, if you want to move forward, where each motor gets the power, set *d* to zero, and *t* to whatever speed you want from 0 to 100%. If you want reverse, again, set *d* to zero, and *t* to the speed you want to go backwards from 0 to -100. To stop the motors, set *d* to zero and *t* to zero.

REMOTE.PY

This module is called when user is in mode-1 (as defined in *globalv.py*). This module is the heart of the remote control function. Upon successful web connection, the module will read *remote.html* into memory and serve it to the user's web browser. *remote.html* also contains some JavaScript that mimics a joystick control. As the user moves the "joystick", it will send updates to the Pico to make the motors move. When the user removes their finger from the joystick, it returns to its central position, and sends a motor stop command, causing the robot to stop any movement. Each pixel of movement is translated to 0-10 to minimize web traffic. The Pico can not handle fast, tiny, data transmission packets over WiFi, thus the joystick movement had to be scaled down. Once the Pico receives the X/Y position data, it is converted to a 0-100% scale and also to a differential drive for each motor. This information is sent to the motor driver, in the case of the *ricmboard*, this will be the *L9110.py* motor driver as two parameters, differential and throttle.

If you view the joystick as four quadrants, anything above the middle is 0-100, meaning motors will move in a forward direction. Anything below the middle will be 0 to ⁻100, which means motors will spin backwards. Anything to the right will be 0 to 100 and result in more speed to the *left* motor, thus pushing the robot to the right. Anything to the left of center will be 0 to-100, resulting in more speed to the *right* motor, thus pushing it to the left.

Several *print()* statements are sprinkled throughout this module to aide in debugging if the variable "*debug*" is set to True in the *globalv* module

This module also frequently check the global variable *button_pressed* located in *globalv.py*. This value will be True if the user has pressed the button for a mode change. The module needs to clean up and exit if the button is pressed. This module will stop the access point (AP), web server (WS), DNS server (DS) and stop all motors before returning back to *main.py*.

PATTERN.PY

This module is called when user is in mode-2 (as defined in *globalv.py*). This mode allows a user to upload at pattern to follow autonomously. Once a file is uploaded, the CarBot will immediately follow this pattern forever, until mode change, or power is removed. If the mode gets changed, the pattern file will be deleted from memory and must be reloaded. This module does several things, first it turns on an access point (AP), a DNS & Web server, so the user must connect to the CarBot via a web browser and upload a file. This can be done from a cell phones, tablet, or computer. This module will validate the pattern file during the upload. If any errors occur in this process, the pattern file upload is aborted, and the line and error will be displayed, allowing the file to be corrected. This module deals a lot with pointers and tables, so be very careful modifyin this module. As each line is sent from the browser, the line is validated for errors, such as invalid commands, command values are out of range, proper formatting, bypassing blank lines or comments, command block definitions, etc. Once the line is validated, it tokenizes the command & any variables, and adds it to the growing internal text line. These tokens consist of index pointers to a command table for later processing.

This module imports *cmd_Gvar.py* which has variable in common to to the validator and runtime functions. The import called *cmd_table.py*, is a table that has all the possible commands that can be used in the pattern table. This table is crucial to validation and operation. On validate, the command, or mnemonic is is checked against his table. If a match is not found, the entire command line is rejected and an error is sent back to the user when uploading. If the mnemonic is found, the index of this table entry is saved, and the rest of the command line is checked for any variables. Each variable has a min/max value, so the command line is checked for variable to be in range. Ie Speed has a variable that sets the throttle speed. It has a min value of -100 to max value of 100. if the user specified a throttle of 256, it would be out of range, and therefore an error and the command line will be rejected. Since this variable can fit into 1 byte of space the command will be tokenized as 1 byte for the pointer into the command table, and byte for the variable. This save memory space and allows for optimized run time.

After the entire pattern file is loaded into memory as a string of tokens, the current pointer fo the command string is set to zero and the pattern will now run. During run time, the byte is pulled from the string and it is used as a pointer into *cmd_table.py* to find the function to process. The *cmd_table.py* is set up as follows:

Elements of cmd_table:

- 1st = Mnemonic code for the command, this is what is used in the pattern text file
- 2nd = Display text for the command, can be used to list a pattern file
- 3rd = Minimum decimal value for the command

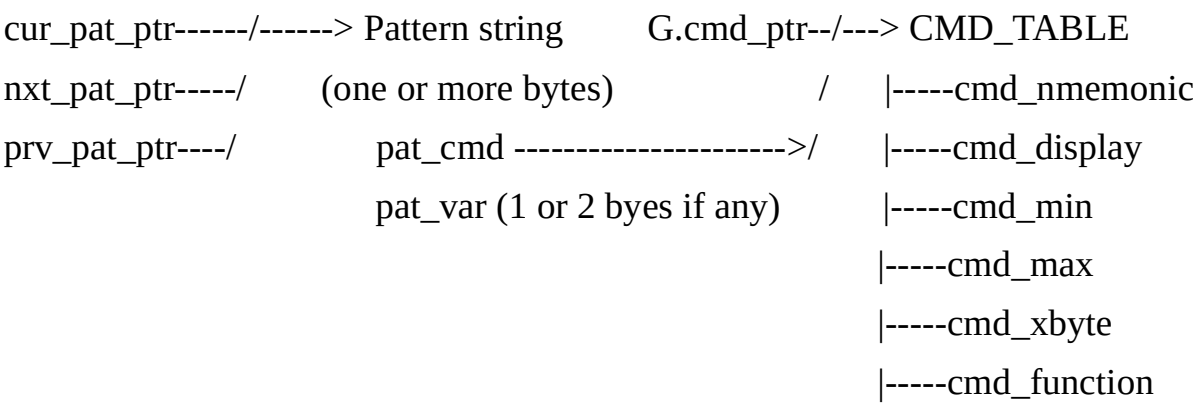
- 4th = Maximum decimal value for the command
- 5th = # of packed bytes for the variable. 0 for no variables for the command
- 6th = Name of function to call in *cmd_funcs.py* !! MUST be defined in *cmd_funcs.py* !!

As specified in the table, the function to run MUST be defined in *cmd_funcs.py*

if there is no function to run, you must specify *f_undefined* to satisfy the run time process. After each process of a command, a function must call *nxt_pat_ptr* so the process knows where the next command to process is located in the pattern string.

A repeat command is being worked on to allow commands to be repeated within a pattern. There is a command that defines up to 10 command blocks with a begin block (BB#) command. Where # is a block number. This allows a user to make blocks of code and run each block with just one command called Execute Block (EB#).

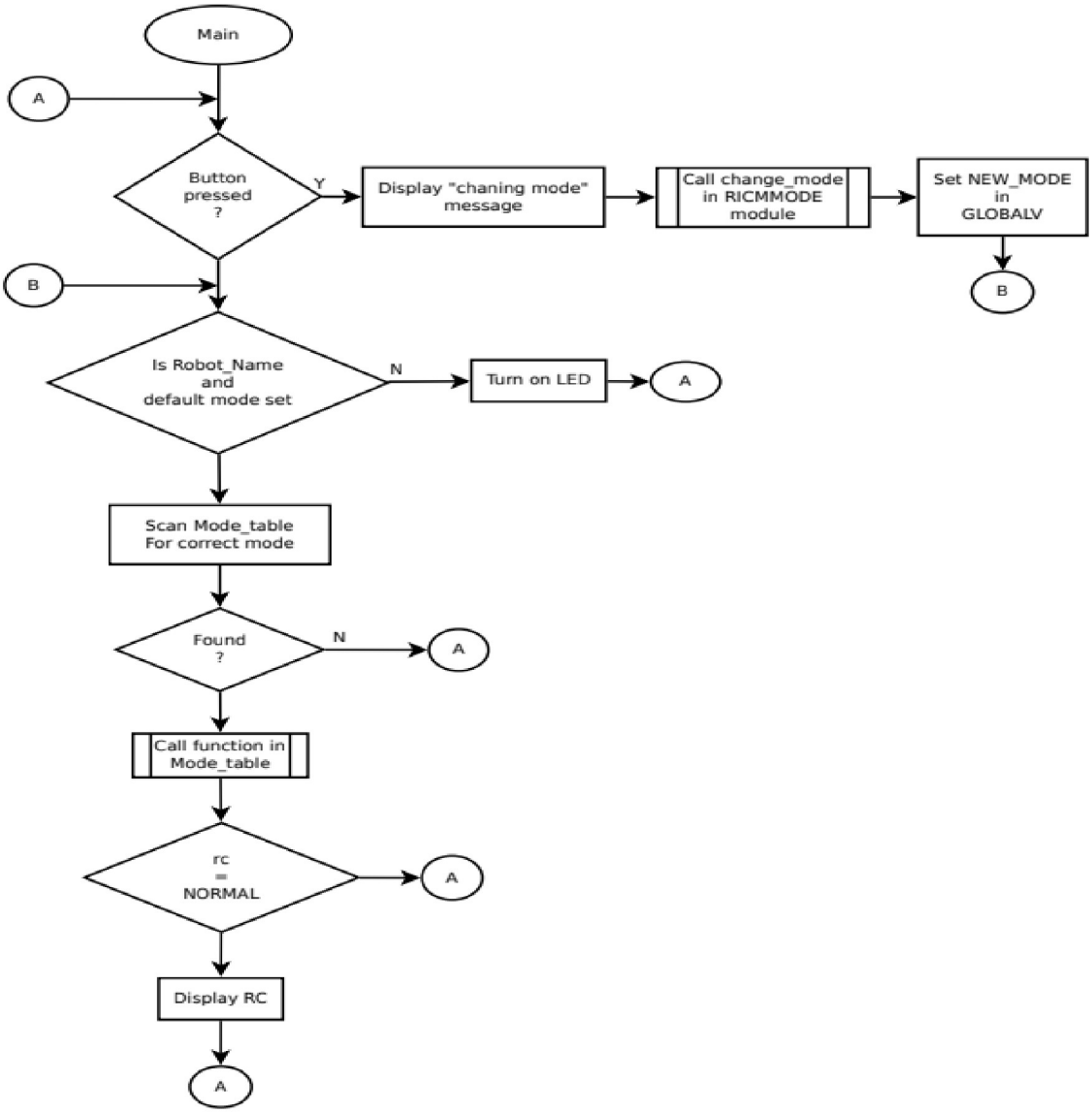
Making tables allows expansion of commands, such as LT with a variable of 0 or 1 meaning to turn of or on a left turn signal.



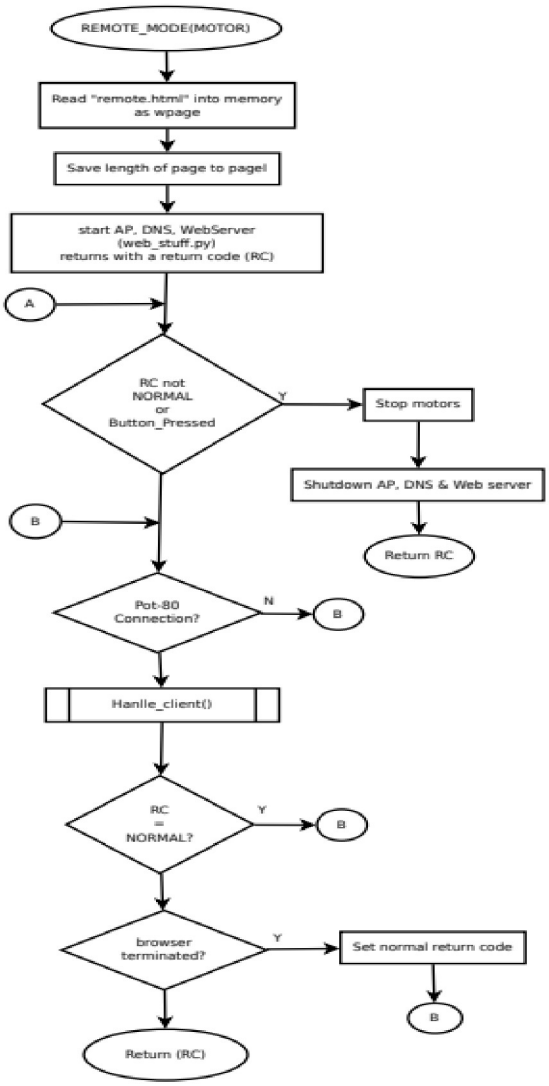
* pat_ptr(s) are merely pointers into the pattern string

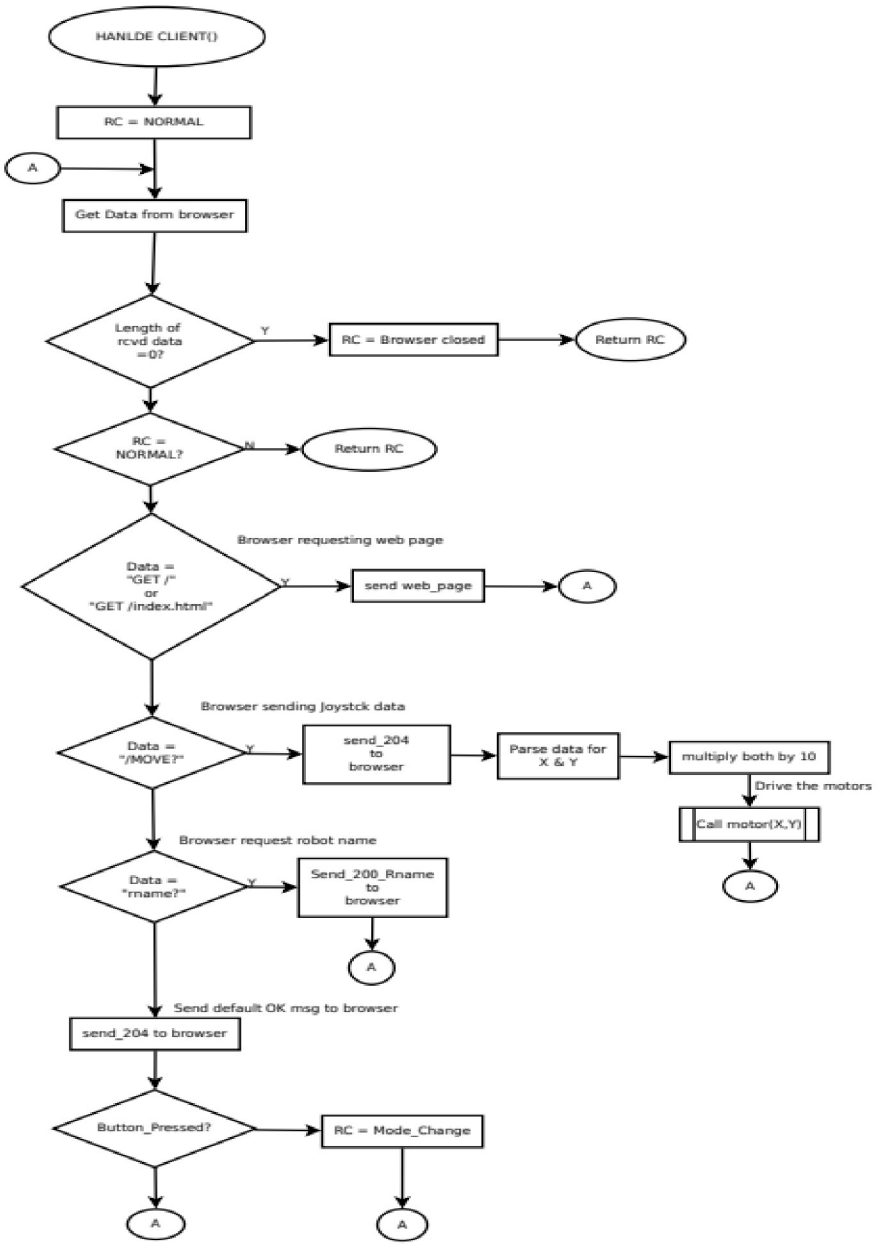
Flow Charts

MAIN.PY

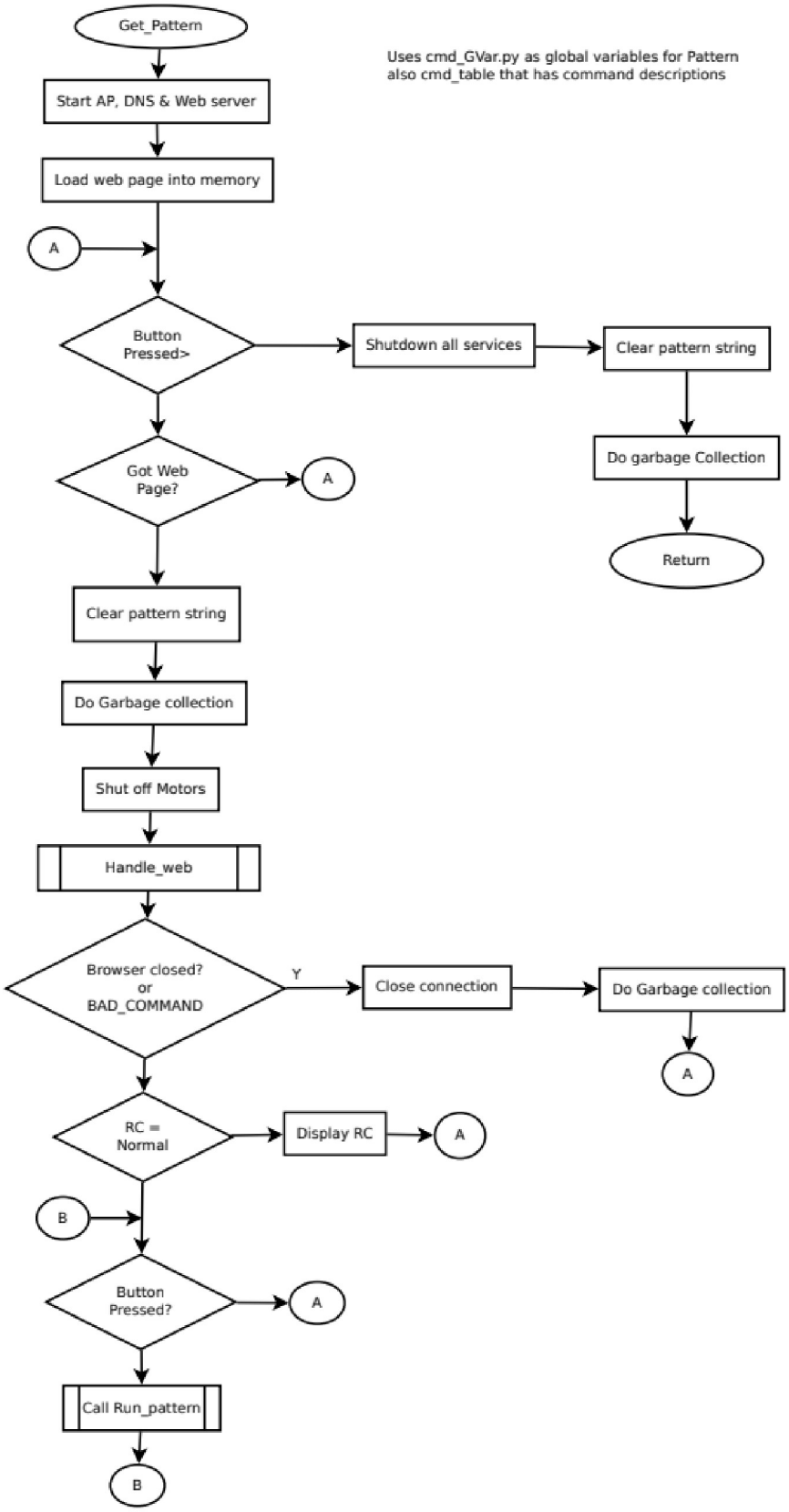


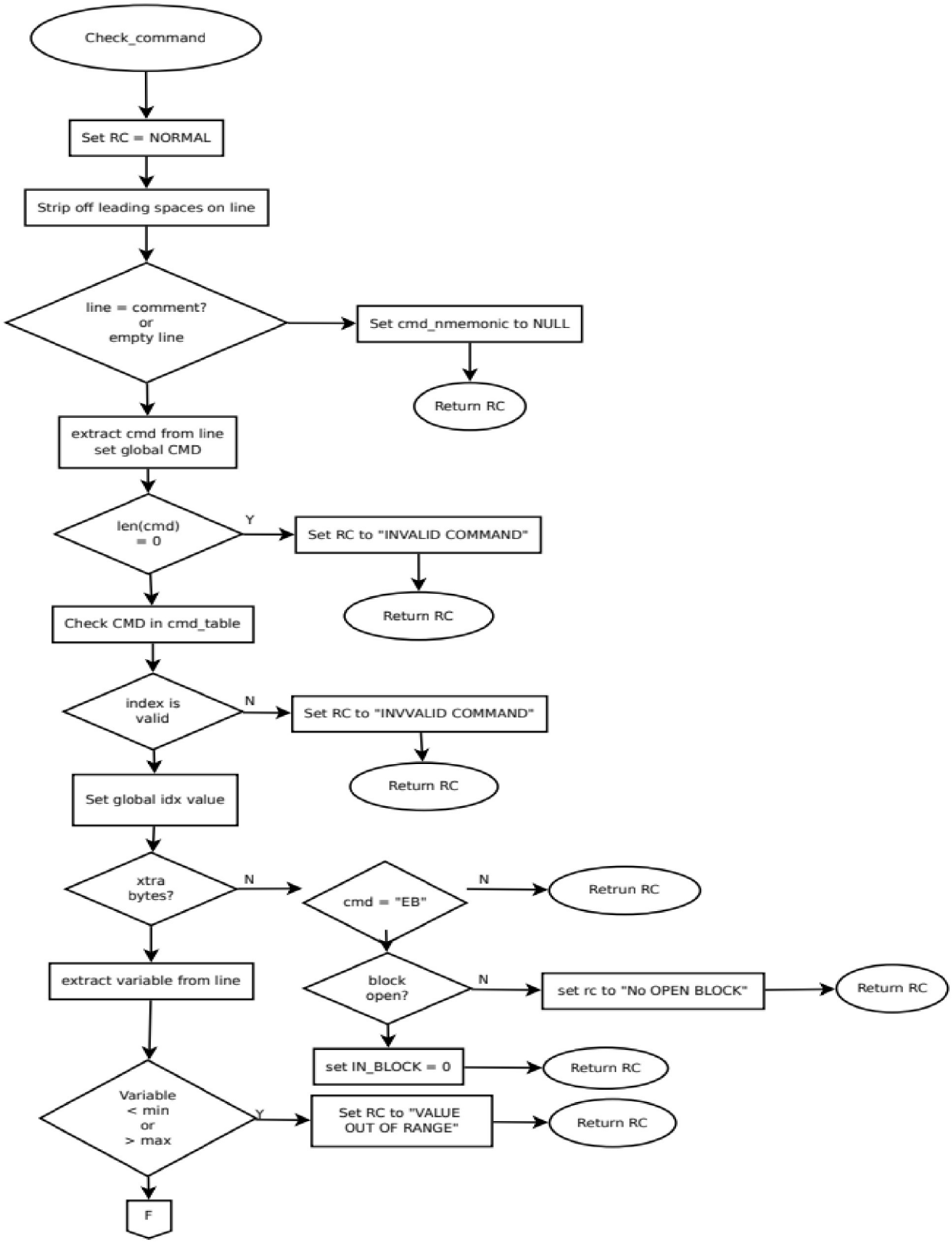
REMOTE.PY

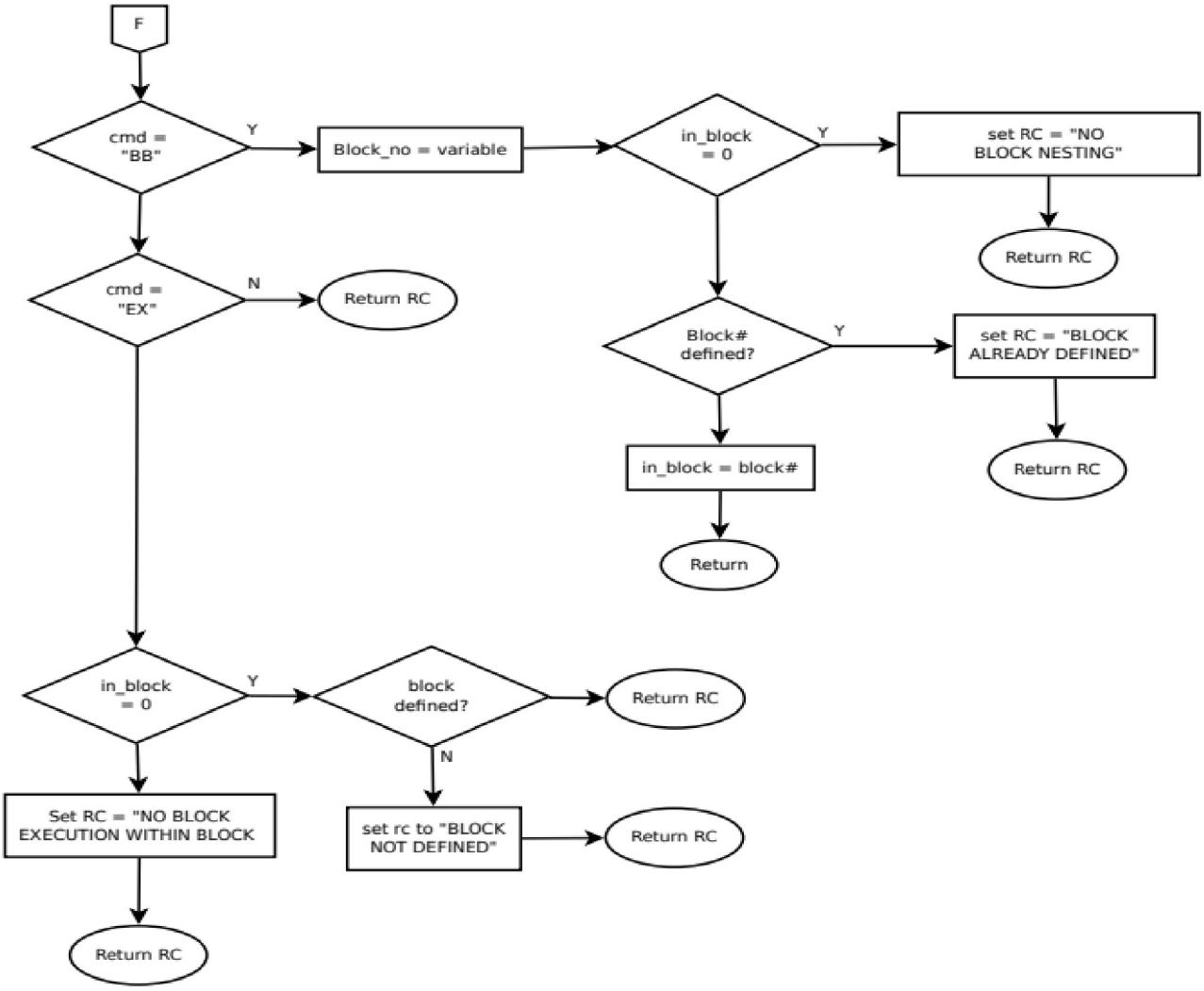

REMOTE_MODE(MOTOR)
Read "remote.html" into memory as wpage
Save length of page to pagel
start AP, DNS, WebServer (web_stuff.py) returns with a return code (RC)
A
RC not NORMAL or Button_Pressed
Y
Stop motors
Shutdown AP, DNS & Web server
Return RC
B
Pot-80 Connection?
N
B
Hanlle_client()
RC = NORMAL?
Y
B
browser terminated?
Y
Set normal return code
B
Return (RC)


HANLDE CLIENT()
RC = NORMAL
A
Get Data from browser
Length of rcvd data =0?
Y
RC = Browser closed
Return RC
RC = NORMAL?
N
Return RC
Browser requesting web page
Data = "GET /" or "GET /index.html"
Y
send web_page
A
Browser sending Joystck data
Data = "/MOVE?"
Y
send_204 to browser
Parse data for X & Y
multiply both by 10
Drive the motors
Call motor(X,Y)
A
Browser request robot name
Data = "rname?"
Y
Send_200_Rname to browser
A
Send default OK msg to browser
send_204 to browser
Button_Pressed?
RC = Mode_Change
A
A

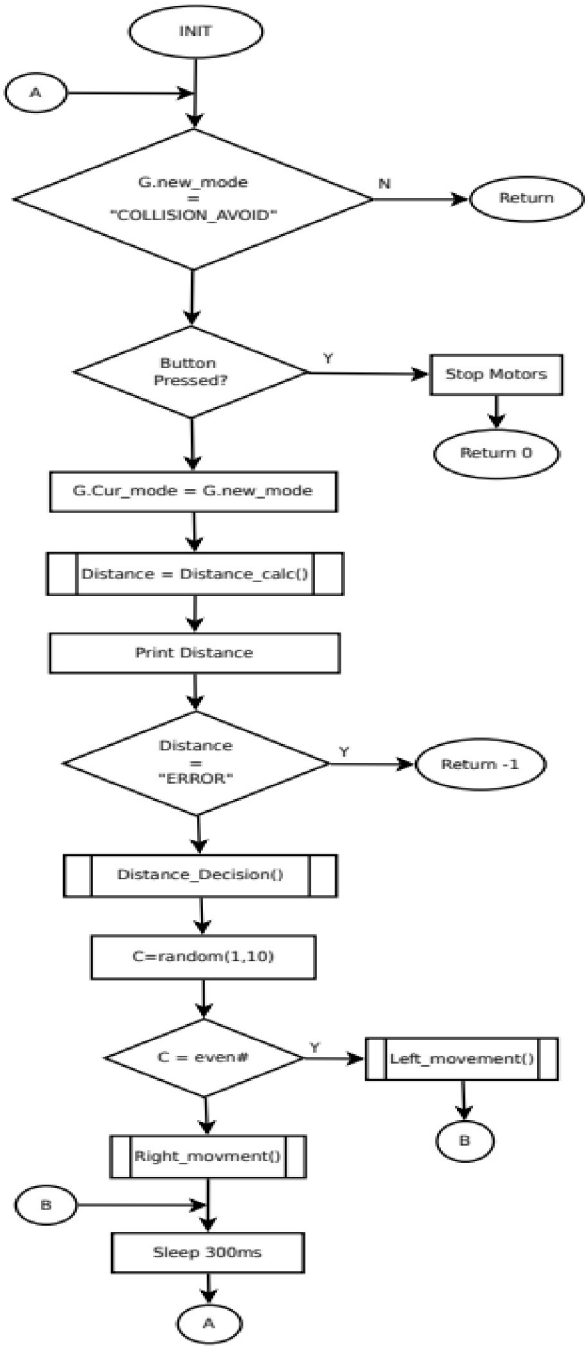
PATTERN.PY

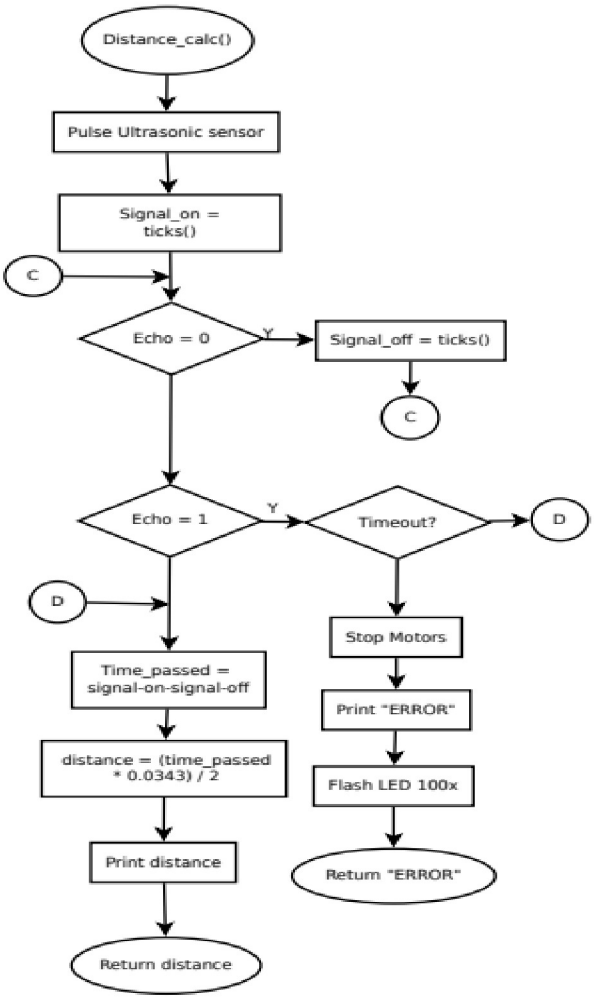

Get_Pattern
Uses cmd_GVar.py as global variables for Pattern
also cmd_table that has command descriptions
Start AP, DNS & Web server
Load web page into memory
A
Button Pressed>
Shutdown all services
Clear pattern string
Do garbage Collection
Return
Got Web Page?
A
Clear pattern string
Do Garbage collection
Shut off Motors
Handle_web
Browser closed? or BAD_COMMAND
Y
Close connection
Do Garbage collection
A
RC = Normal
Display RC
A
B
Button Pressed?
A
Call Run_pattern
B

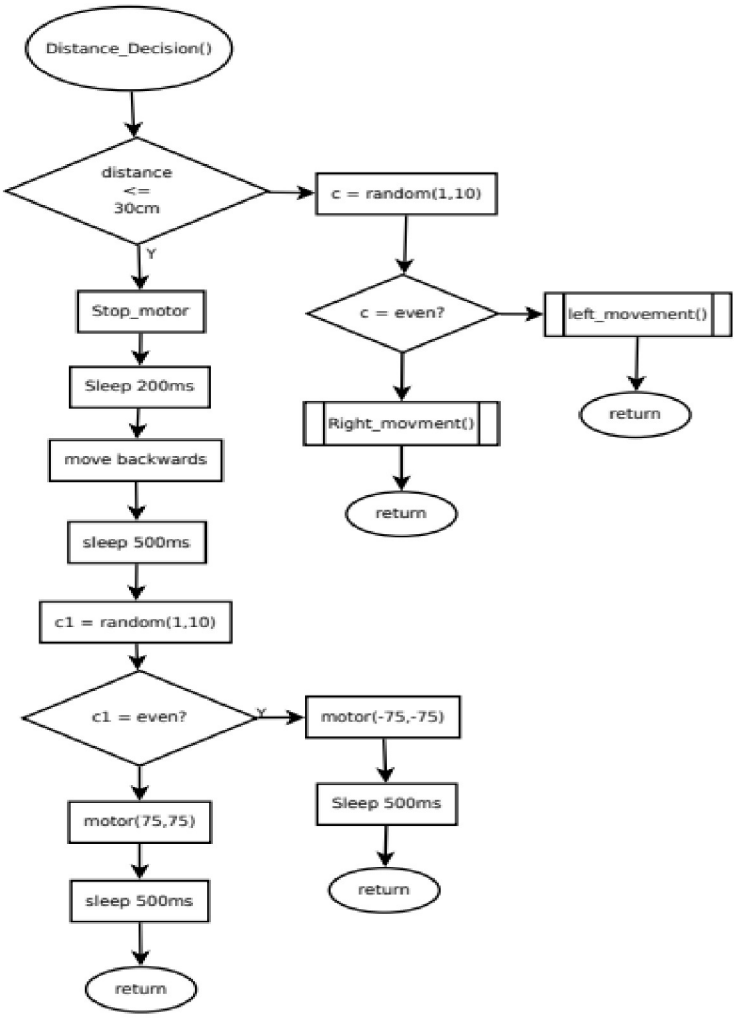

Check_command
Set RC = NORMAL
Strip off leading spaces on line
line = comment?
or
empty line
Y
Set cmd_nmemonic to NULL
Return RC
extract cmd from line
set global CMD
len(cmd)
= 0
Y
Set RC to "INVALID COMMAND"
Return RC
Check CMD in cmd_table
index is
valid
N
Set RC to "INVVALID COMMAND"
Return RC
Set global idx value
xtra
bytes?
N
cmd = "EB"
N
Retrun RC
extract variable from line
block
open?
N
set rc to "No OPEN BLOCK"
Return RC
set IN_BLOCK = 0
Return RC
Variable
< min
or
> max
Y
Set RC to "VALUE
OUT OF RANGE"
Return RC
F

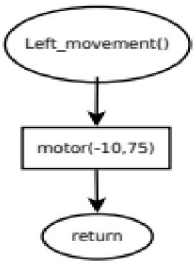

F
cmd = "BB"
Y
Block_no = variable
in_block = 0
Y
set RC = "NO BLOCK NESTING"
Return RC
cmd = "EX"
N
Return RC
Block# defined?
Y
set RC = "BLOCK ALREADY DEFINED"
Return RC
in_block = block#
Return
in_block = 0
Y
block defined?
Return RC
N
Set RC = "NO BLOCK EXECUTION WITHIN BLOCK
set rc to "BLOCK NOT DEFINED"
Return RC
Return RC

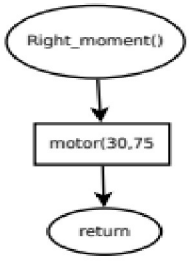
COLLISION.PY






Distance_Decision()
distance
<=
30cm
Y
Stop_motor
Sleep 200ms
move backwards
sleep 500ms
c1 = random(1,10)
c1 = even?
Y
motor(75,75)
sleep 500ms
return
motor(-75,-75)
Sleep 500ms
return
c = random(1,10)
c = even?
left_movement()
return
Right_movment()
return


Left_movement()
motor(-10,75)
return


Right_moment()
motor(30,75
return